

Computer Vision & Deep Learning II

CMSC 178IP - Digital Image Processing

Noel Jeffrey Pinton

University of the Philippines - Cebu
Department of Computer Science

Course Outline

Image Classification

Image Segmentation

Object Detection

Best Practices and Applications

Image Classification

Image Classification Overview

Definition

Image classification assigns a single label to an entire image from a predefined set of categories.

Core Task:

- Input: Image $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$
- Output: Class label $y \in \{1, 2, \dots, K\}$
- Goal: Learn mapping $f : \mathbf{I} \rightarrow y$

Applications:

- Medical diagnosis (X-ray, MRI classification)
- Quality control in manufacturing
- Content moderation and filtering
- Autonomous vehicles (traffic sign recognition)

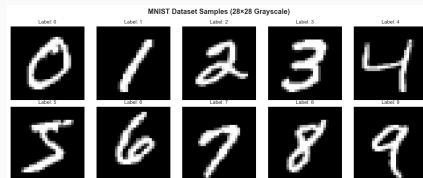
MNIST Dataset

Dataset Properties:

- 70,000 handwritten digits (0-9)
- 60,000 training, 10,000 test images
- 28×28 grayscale images
- Centered and size-normalized

Challenges:

- Writing style variations
- Digit similarity (1 vs 7, 3 vs 8)
- Image noise and artifacts



Sample MNIST digits

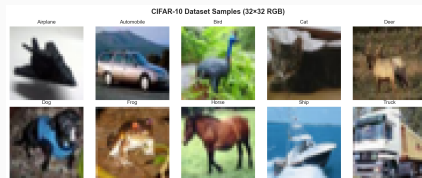
CIFAR-10 Dataset

Dataset Properties:

- 60,000 color images in 10 classes
- 50,000 training, 10,000 test
- 32×32 RGB images
- Real-world natural images

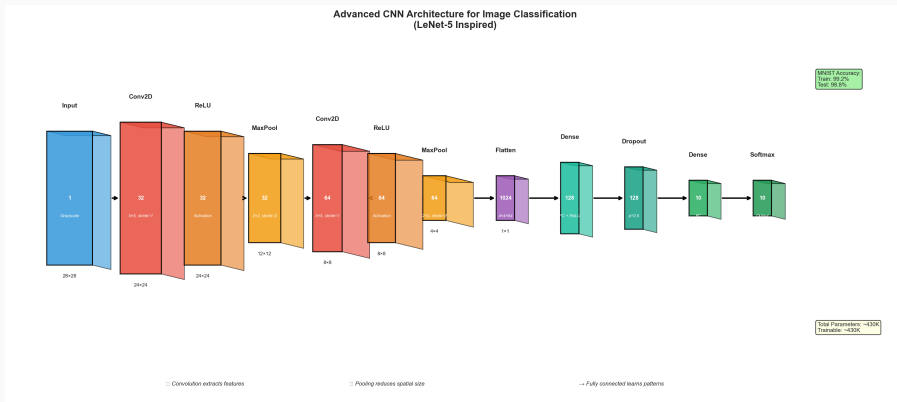
Classes:

- Airplane, Automobile, Bird, Cat, Deer
- Dog, Frog, Horse, Ship, Truck



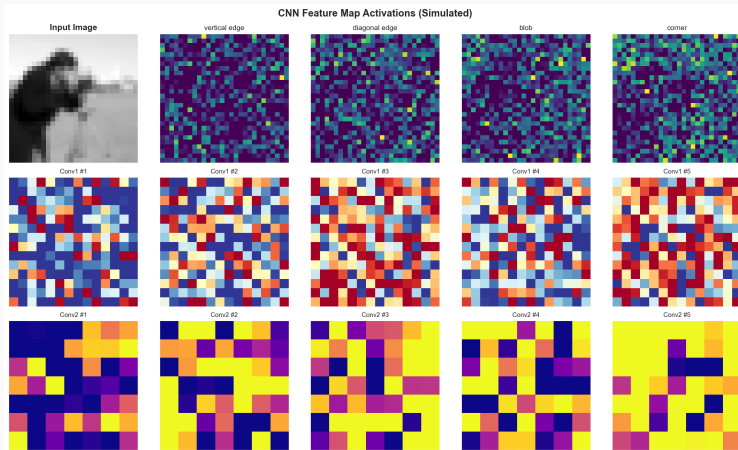
CIFAR-10 examples

CNN Architecture for Classification



Components: Conv (features) → Pool (downsample) → FC (classify) → Softmax

Feature Map Visualization



Early layers detect edges/textures, deeper layers learn complex patterns

Loss Function: Cross-entropy

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik}) \quad (1)$$

Training: Forward pass $\rightarrow$ Loss $\rightarrow$ Backprop $\rightarrow$ Update weights

Key Hyperparameters:

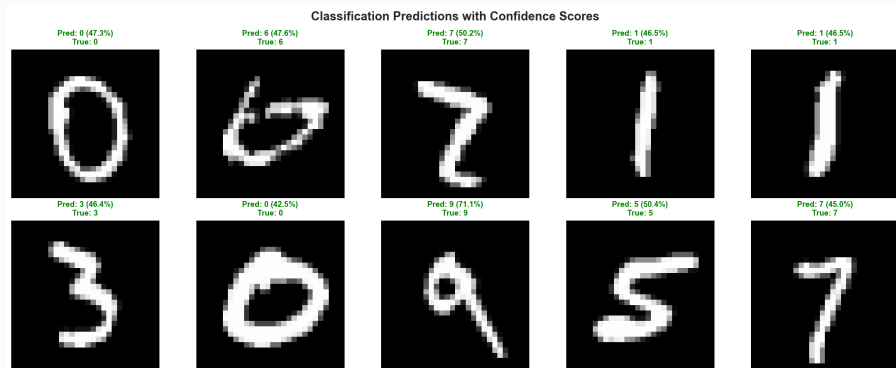
- Learning rate: 10^{-3} to 10^{-4} , Batch: 32-128
- Optimizer: SGD, Adam, RMSprop

Training Curves



Monitoring: Train accuracy $\uparrow$, val accuracy shows generalization, gap = overfitting

Model Predictions



Analysis: Confidence scores, correct vs incorrect predictions, border colors show accuracy

Data Augmentation

Purpose: Increase training data diversity and reduce overfitting

Common Techniques:

- **Geometric:** Rotation, flipping, translation, scaling
- **Color:** Brightness, contrast, saturation adjustments
- **Noise:** Gaussian noise, random erasing
- **Advanced:** Cutout, Mixup, AutoAugment

Implementation:

- Apply transformations on-the-fly during training
- Preserve label information
- Use library functions (torchvision, Albumentations)

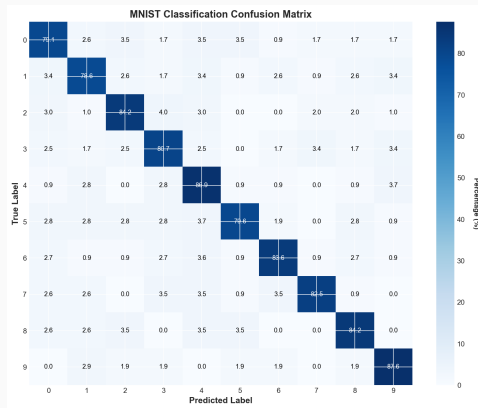
Evaluation Metrics

Key Metrics:

- **Accuracy:** Overall correctness
- **Precision:** True positives / predicted positives
- **Recall:** True positives / actual positives
- **F1 Score:** Harmonic mean of precision and recall

Per-Class Analysis:

- Confusion matrix visualization
- Identify challenging classes
- Class imbalance detection



Confusion matrix example

Classic CNN Architectures

LeNet-5 (1998):

- First successful CNN for digit recognition
- 7 layers, pioneered conv-pool-fc structure

AlexNet (2012):

- ImageNet breakthrough, 8 layers
- ReLU activation, dropout regularization

VGGNet (2014):

- Very deep (16-19 layers), small 3×3 filters
- Demonstrated depth importance

ResNet (2015):

- Skip connections enable very deep networks (50-152 layers)
- Solved vanishing gradient problem

Image Segmentation

Segmentation Overview

Definition

Image segmentation partitions an image into multiple segments or regions, assigning a label to every pixel.

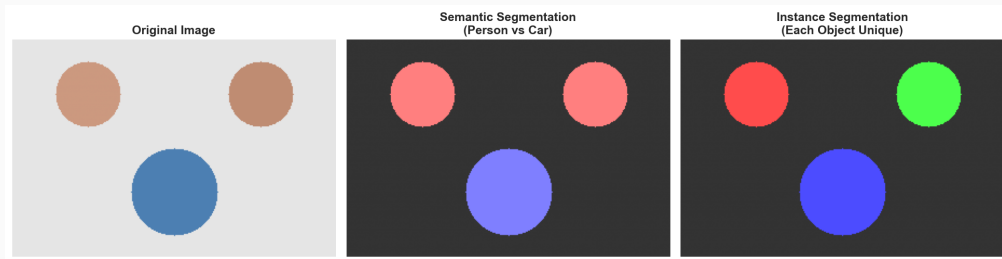
Segmentation Types:

- **Semantic:** Classify each pixel by category
- **Instance:** Distinguish individual objects
- **Panoptic:** Combine semantic and instance

Applications:

- Medical imaging (tumor detection, organ segmentation)
- Autonomous driving (road, pedestrian, vehicle detection)
- Satellite imagery (land cover classification)
- Video editing and AR/VR

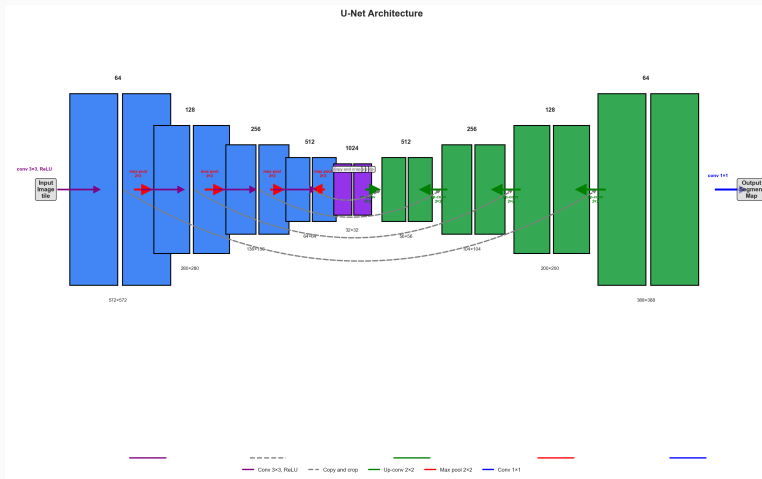
Semantic vs Instance Segmentation



Key Differences:

- **Semantic:** Same class → same label (all persons labeled "person")
- **Instance:** Each object gets unique ID (person1, person2, ...)
- Instance segmentation is more challenging but provides richer information

U-Net Architecture



Encoder-decoder + skip connections + symmetry

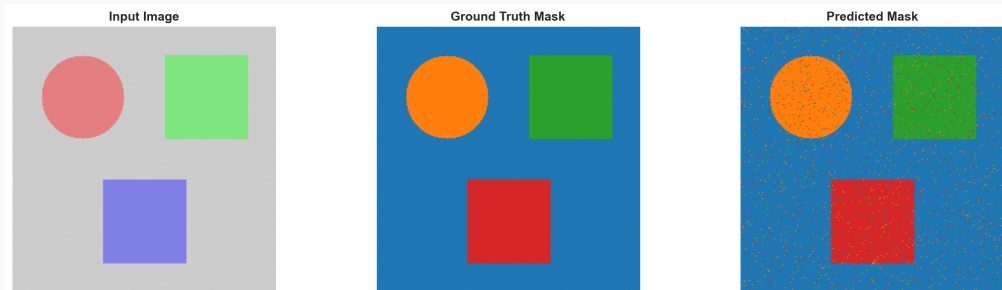
U-Net Components

Encoder: Conv + ReLU + pooling, doubles channels, halves resolution

Decoder: Upconv + concatenate encoder features (skip connections)

Output: 1×1 conv $\rightarrow$ pixel-wise softmax

Segmentation Example



Workflow:

1. Input image fed through network
2. Model predicts class for each pixel
3. Compare prediction with ground truth
4. Compute loss and update weights

Segmentation Loss Functions

Cross-Entropy Loss:

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic}) \quad (2)$$

Dice Loss:

$$\mathcal{L}_{Dice} = 1 - \frac{2 \sum_{i=1}^N p_i g_i}{\sum_{i=1}^N p_i + \sum_{i=1}^N g_i} \quad (3)$$

Choosing Loss:

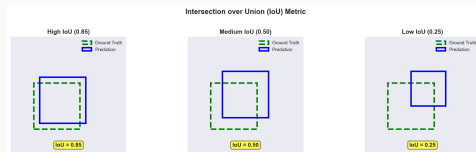
- Cross-entropy: Standard choice, class-balanced
- Dice: Better for imbalanced data, small objects
- Combined: Often use both for robustness

Intersection over Union (IoU):

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \quad (4)$$

Mean IoU (mIoU):

- Average IoU across all classes
- Standard metric for segmentation
- Range: $[0, 1]$, higher is better



IoU examples

Advanced Segmentation Architectures

FCN (Fully Convolutional Network):

- First end-to-end CNN for segmentation
- Replace FC layers with convolutions

SegNet:

- Encoder-decoder with pooling indices
- Memory efficient upsampling

DeepLab:

- Atrous/dilated convolutions for larger receptive field
- ASPP (Atrous Spatial Pyramid Pooling)

Mask R-CNN:

- Extends Faster R-CNN for instance segmentation
- Adds mask prediction branch

Object Detection

Object Detection Overview

Definition

Object detection identifies and localizes multiple objects in an image, providing both class labels and bounding box coordinates.

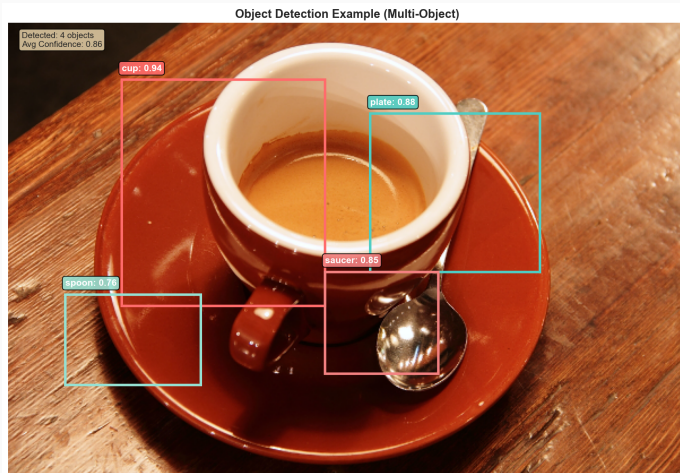
Output Format:

- **Bounding box:** (x, y, w, h) coordinates
- **Class label:** Object category
- **Confidence score:** Detection certainty

Applications:

- Autonomous vehicles (pedestrian, vehicle detection)
- Surveillance and security systems
- Retail (product recognition, inventory)
- Robotics (object manipulation)

Object Detection Example



Challenges: Variable count, scales, occlusion, real-time

Two Approaches to Object Detection

Two-Stage Detectors:

- Step 1: Generate region proposals
- Step 2: Classify and refine proposals
- Examples: R-CNN, Fast R-CNN, Faster R-CNN
- Higher accuracy, slower inference

One-Stage Detectors:

- Direct prediction of boxes and classes
- Single network pass
- Examples: YOLO, SSD, RetinaNet
- Faster inference, slightly lower accuracy

R-CNN Family Evolution

R-CNN (2014):

- Selective search for proposals (2000 per image)
- CNN features + SVM classifier
- Slow: 47s per image

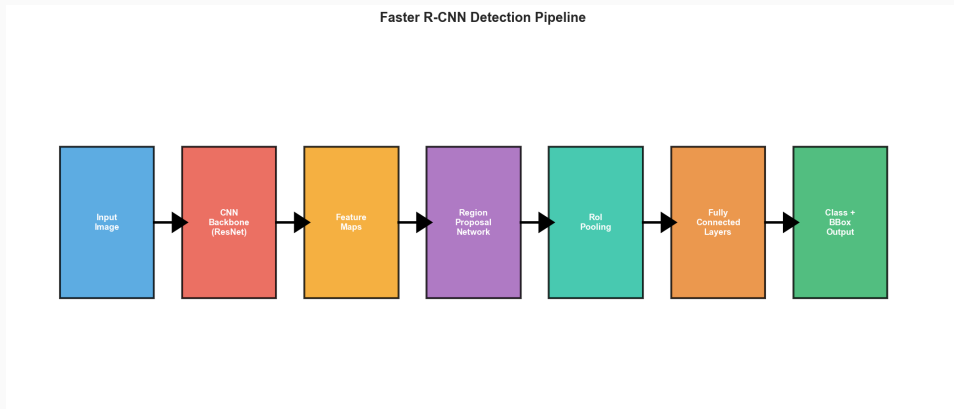
Fast R-CNN (2015):

- Single CNN for entire image
- RoI pooling for feature extraction
- Faster: 2s per image

Faster R-CNN (2015):

- Region Proposal Network (RPN)
- End-to-end trainable
- Real-time capable: 0.2s per image

Faster R-CNN Architecture



Pipeline: CNN backbone → RPN → RoI pooling → Detection head

Region Proposal Network (RPN)

Concept: Slides over feature map, predicts objectness

Anchor Boxes: 9 per location ($3 \text{ scales} \times 3 \text{ ratios}$)

Outputs: Objectness score + bbox offsets

YOLO: You Only Look Once

Key Idea

YOLO treats detection as a single regression problem, directly predicting bounding boxes and class probabilities from full images in one evaluation.

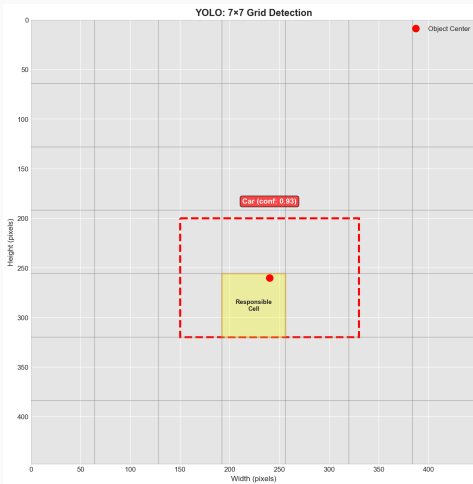
Advantages:

- Extremely fast: 45+ FPS on GPU
- Global context (sees entire image)
- Fewer false positives on background

Architecture:

- Divides image into $S \times S$ grid
- Each cell predicts B bounding boxes
- Each box has: $(x, y, w, h, \text{confidence})$
- Plus C class probabilities per cell

YOLO Grid-Based Detection



Cell with object center predicts bbox + class

Multi-part Loss:

$$\mathcal{L} = \lambda_{coord}\mathcal{L}_{loc} + \mathcal{L}_{conf} + \lambda_{noobj}\mathcal{L}_{noobj} + \mathcal{L}_{class} \quad (5)$$

Components: Localization + confidence + classification

Weights: $\lambda_{coord} = 5$ (emphasize localization), $\lambda_{noobj} = 0.5$

YOLOv1 (2016):

- Original grid-based approach
- 45 FPS, 63.4% mAP on VOC

YOLOv2/YOLO9000 (2017):

- Batch normalization, anchor boxes
- Multi-scale training

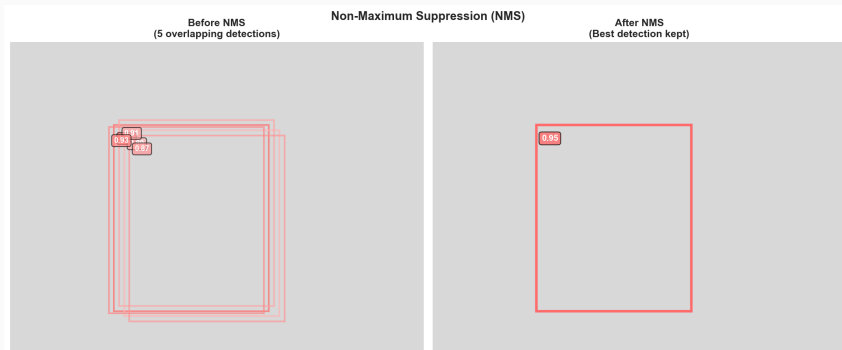
YOLOv3 (2018):

- Feature pyramid, better small object detection
- Logistic regression for classes

YOLOv4-v8 (2020-2023):

- CSPNet backbone, advanced augmentation
- State-of-the-art speed/accuracy trade-off

Non-Maximum Suppression (NMS)



Goal: Remove duplicate detections

Method: Sort by confidence, keep highest, remove overlaps ($\text{IoU} > \text{threshold}$)

Evaluation: Mean Average Precision (mAP)

Average Precision (AP):

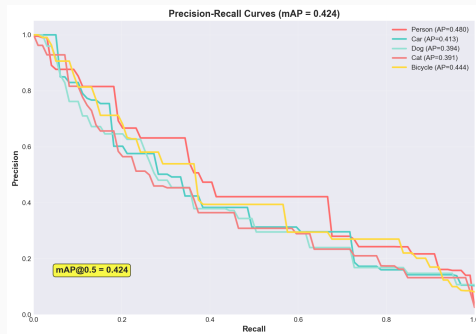
- Area under precision-recall curve
- Computed per class
- Varies with IoU threshold

Mean AP (mAP):

$$\text{mAP} = \frac{1}{C} \sum_{c=1}^C \text{AP}_c \quad (6)$$

Standard Metrics:

- mAP@0.5: IoU threshold 0.5
- mAP@[0.5:0.95]: Average over multiple IoU thresholds



Precision-recall curves

Detection Challenges

Scale Variation:

- Objects appear at different sizes
- Solution: Feature pyramids, multi-scale training

Occlusion:

- Objects partially hidden
- Solution: Context modeling, attention mechanisms

Class Imbalance:

- Many more background samples
- Solution: Focal loss, hard negative mining

Real-time Constraints:

- Need fast inference for applications
- Solution: Lightweight backbones, pruning, quantization

Best Practices and Applications

Key Concept

Transfer learning leverages pretrained models on large datasets (ImageNet) and fine-tunes them for specific tasks.

Advantages:

- Faster training convergence
- Better performance with limited data
- Learned general features transfer well

Strategies:

- **Feature extraction:** Freeze early layers, train only classifier
- **Fine-tuning:** Train entire network with small learning rate
- **Progressive:** Gradually unfreeze layers during training

Model Selection Guidelines

For Classification:

- Small datasets: ResNet-18, MobileNet
- Large datasets: ResNet-50/101, EfficientNet
- Limited compute: SqueezeNet, MobileNetV3

For Segmentation:

- Medical imaging: U-Net, U-Net++
- General purpose: DeepLabv3+, HRNet
- Real-time: ENet, BiSeNet

For Detection:

- High accuracy: Faster R-CNN, Cascade R-CNN
- Real-time: YOLOv5/v8, SSD
- Mobile devices: MobileNet-SSD, YOLO-tiny

Hyperparameter Tuning

Learning Rate:

- Start: 10^{-3} to 10^{-4}
- Use learning rate finder or schedules
- Cosine annealing, step decay

Batch Size:

- Larger batches: More stable, needs more memory
- Smaller batches: Noisy gradients, regularization effect
- Typical: 16-64 for detection, 32-128 for classification

Regularization:

- Dropout: 0.2-0.5 typical
- Weight decay: 10^{-4} to 10^{-5}
- Early stopping based on validation loss

Check Data Pipeline:

- Visualize augmented samples
- Verify label correctness
- Check normalization statistics

Monitor Training:

- Plot loss curves (train and validation)
- Track gradient magnitudes
- Visualize predictions on validation set

Overfit on Small Batch:

- Try to overfit 1-10 samples
- If fails, check architecture or learning rate

Common Issues:

- Exploding/vanishing gradients: Check initialization, use batch norm
- Poor convergence: Adjust learning rate, check data quality
- Overfitting: More data, stronger regularization, data augmentation

Medical Imaging Application

Use Case: Lung nodule detection in CT scans

Approach:

- U-Net for nodule segmentation
- Transfer learning from ImageNet
- Data augmentation: rotation, scaling, elastic deformation
- Ensemble of models for robustness

Challenges:

- Class imbalance (few positive samples)
- 3D volumetric data
- High precision requirements
- Limited labeled data

Use Case: Multi-task perception system

Tasks:

- Object detection (pedestrians, vehicles, cyclists)
- Lane segmentation
- Drivable area segmentation
- Traffic sign classification

Architecture:

- Shared backbone for efficiency
- Multi-task learning with task-specific heads
- Real-time inference (>30 FPS)

Image Classification:

- CNNs learn hierarchical features
- Data augmentation and transfer learning are essential
- Monitor training/validation curves

Image Segmentation:

- Pixel-wise classification for dense prediction
- U-Net and encoder-decoder architectures
- IoU and Dice metrics for evaluation

Object Detection:

- Two-stage (Faster R-CNN) vs one-stage (YOLO)
- Trade-off between speed and accuracy
- NMS and mAP for post-processing and evaluation

Important Points

Deep learning has revolutionized computer vision, but success requires careful attention to data quality, architecture selection, and training procedures.

Best Practices:

- Start with pretrained models (transfer learning)
- Use data augmentation extensively
- Monitor validation metrics to prevent overfitting
- Select architecture based on task requirements
- Iterate on data quality and model design

Questions?